

الگوریتم‌های گراف

نمایش گراف

فرض کنید که گراف $G = (V, E)$ داده شده است. در شبه‌کد، مجموعه رأس‌ها را با $G.V$ و مجموعه یال‌ها را با $G.E$ نشان می‌دهیم.

- G می‌تواند جهت‌دار یا بدون جهت باشد.
- دو روش متداول برای نمایش گراف‌ها برای الگوریتم‌ها:
 - لیست‌های مجاورت.
 - ماتریس مجاورت.

هنگام بیان زمان اجرای یک الگوریتم، معمولاً بر حسب هر دو $|V|$ و $|E|$ بیان می‌شود.

در زمان بیان پیچیدگی با استفاده از نمادها می‌توانیم علامت کاردینالتی $||$ را حذف کرد.

مثال: $O(V + E)$ در واقع به معنای $O(|V| + |E|)$ است.

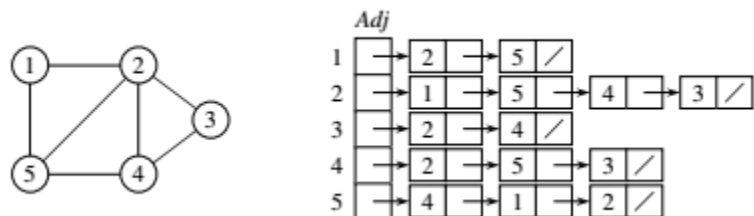
نمایش گراف با لیست‌های مجاورت

آرایه Adj شامل $|V|$ لیست، یکی برای هر رأس.

لیست رأس u شامل تمام رأس‌هایی مانند v است به طوری که $(u, v) \in E$. (برای هر دو گراف جهت‌دار و بدون جهت کار می‌کند).

در شبه‌کد، آرایه را به عنوان ویژگی $G.Adj$ نشان می‌دهیم، بنابراین نمادهایی مانند $G.Adj[u]$ را خواهیم دید.

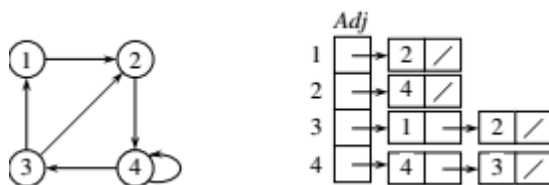
مثال: برای یک گراف بدون جهت:



- اگر یال‌ها وزن داشته باشند، می‌توان وزن‌ها را در لیست‌ها قرار داد. $w: E \rightarrow \mathbb{R}$

- بعداً از وزن‌ها برای درخت‌های پوشا و کوتاه‌ترین مسیرها استفاده خواهیم کرد. $\Theta(V + E)$ فضا:
- زمان: برای لیست کردن تمام رأس‌های مجاور u : $\Theta(\text{degree}(u))$.
- زمان: برای تعیین اینکه آیا $(u, v) \in E$ است: $O(\text{degree}(u))$.

مثال: نمایش لیست مجاورت برای یک گراف جهتدار



نمایش گراف به کمک ماتریس مجاورت

$|V| \times |V|$ matrix $A = (a_{ij})$

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

	1	2	3	4
1	0	1	0	0
2	0	0	0	1
3	1	1	0	0
4	0	0	1	1

- فضا مورد نیاز برای ذخیره ماتریس مجاورت: $\Theta(V^2)$.
- زمان: برای لیست کردن تمام رأس‌های مجاور u : $\Theta(V)$.
- زمان: برای تعیین اینکه آیا $(u, v) \in E$ است: $\Theta(1)$.

می‌توان به جای بیت‌ها، وزن‌ها را برای گراف وزن‌دار ذخیره کرد.

نمایش ویژگی‌های گراف

- الگوریتم‌های گراف معمولاً نیاز به حفظ ویژگی‌هایی برای رأس‌ها و/یا یال‌ها دارند. از نماد نقطه‌ای معمول استفاده کنید: ویژگی d رأس v را با $v.d$ نشان دهید.

- برای یال‌ها نیز از نماد نقطه‌ای استفاده کنید: ویژگی f یال (u, v) را با $f(u, v)$ نشان دهید.

پیاده‌سازی ویژگی‌های گراف

- هیچ روش بهینه واحدی برای پیاده‌سازی وجود ندارد. این امر به زبان برنامه‌نویسی، الگوریتم و نحوه تعامل بقیه برنامه با گراف بستگی دارد.
- اگر گراف با لیست‌های مجاورت نمایش داده شود، می‌توان ویژگی‌های رأس‌ها را در آرایه‌های اضافی موازی با آرایه Adj نمایش داد، مثلاً $d[1:|V|]$. به طوری که اگر رأس‌های مجاور u در Adj[u] باشند، مقدار $u.d$ در خلنه آرایه $d[u]$ ذخیره شود.
- اما می‌توان ویژگی‌ها را به روش‌های دیگر نیز نمایش داد. مثال: می‌توان ویژگی‌های رأس‌ها را به عنوان متغیرهای نمونه در یک زیرکلاس از کلاس Vertex نمایش داد.

جستجوی اول سطح

- ورودی:
 - گراف $G = (V, E)$ ، جهت‌دار یا بدون جهت، و رأس مبدأ $s \in V$.
- خروجی:
 - $d.v$ = فاصله (کمترین تعداد یال) از s تا v ، برای همه $v \in V$.
 - $\pi.v$ رأس قبل از v در کوتاه‌ترین مسیر (کمترین تعداد یال) از s است.
 - (u, v) آخرین یال در کوتاه‌ترین مسیر $v \rightsquigarrow s$ است.
 - زیرگراف پیشینی (درخت جستجوی سطحی)
 - شامل یال‌های (u, v) است به طوری که $\pi.v = u$.
 - زیرگراف پیشینی یک درخت تشکیل می‌دهد، به نام درخت جستجوی اول سطح.

شهود

- جستجوی اول سطح مرز بین رأس‌های کشف شده و کشف نشده را به طور یکنواخت در سراسر عرض مرز گسترش می‌دهد.
- رأس‌ها را به صورت موج‌هایی کشف می‌کند. برای این کار با شروع از رأس S :
 - ابتدا تمام رأس‌هایی که ۱ یال از S فاصله دارند را بازدید می‌کند.
 - از آنجا، تمام رأس‌هایی که ۲ یال از S فاصله دارند را بازدید می‌کند.
 - و غیره.

از یک صف FIFO مانند Q برای حفظ جبهه موج استفاده کنید.

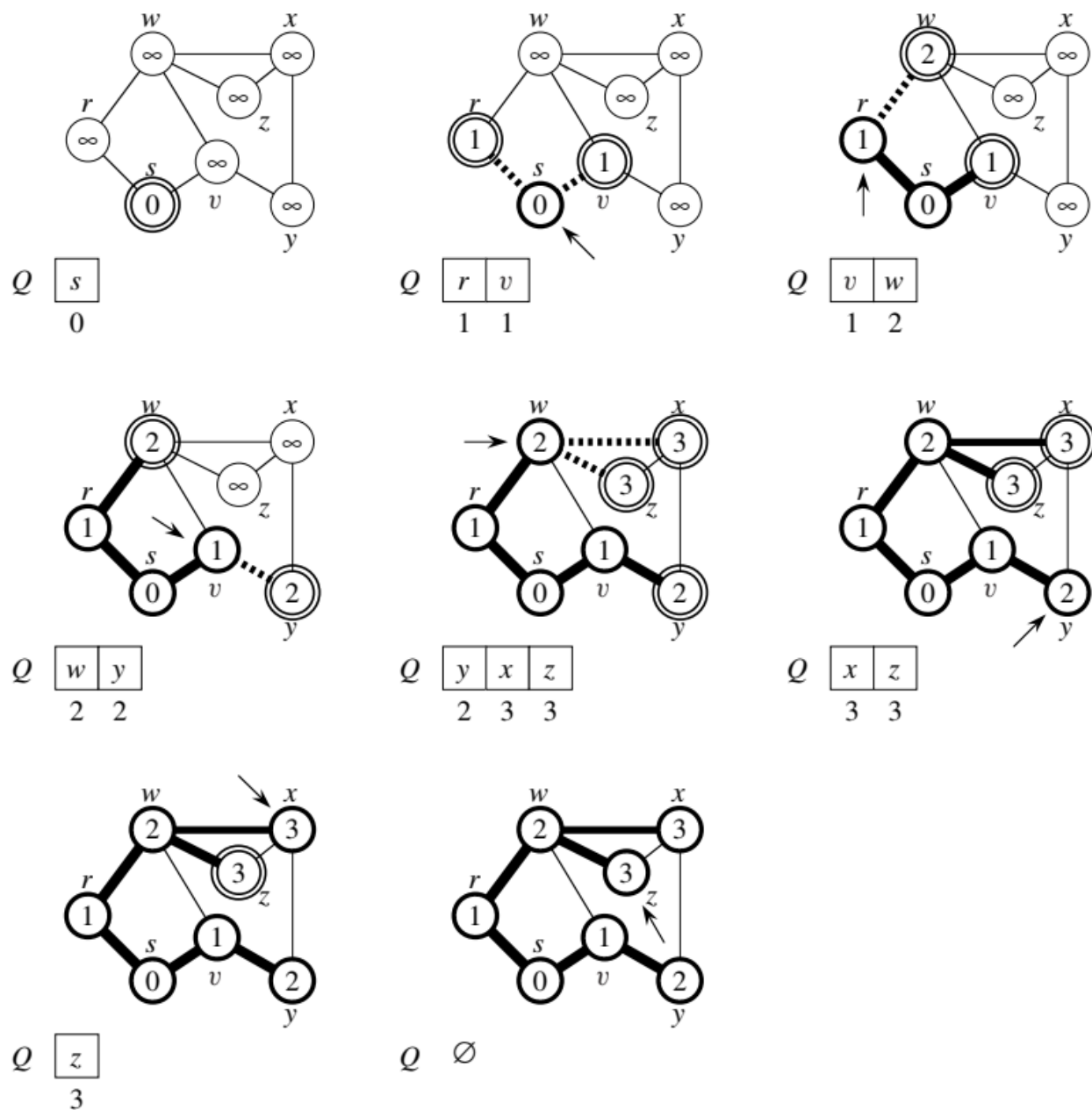
- اگر $v \in Q$ و فقط اگر موج v را بازدید کرده باشد اما هنوز از v خارج نشده باشد.
- Q شامل رأس‌هایی با فاصله k است، و احتمالاً برخی رأس‌ها با فاصله $k + 1$. بنابراین، در هر زمان Q بخش‌هایی از دو موج متوالی را شامل می‌شود.

```

BFS( $V, E, s$ )
  for each vertex  $u \in V - \{s\}$ 
     $u.d = \infty$ 
     $u.\pi = \text{NIL}$ 
   $s.d = 0$ 
   $Q = \emptyset$ 
  ENQUEUE( $Q, s$ )
  while  $Q \neq \emptyset$ 
     $u = \text{DEQUEUE}(Q)$ 
    for each vertex  $v$  in  $G.\text{Adj}[u]$  // search the neighbors of  $u$ 
      if  $v.d == \infty$  // is  $v$  being discovered now?
         $v.d = u.d + 1$ 
         $v.\pi = u$ 
        ENQUEUE( $Q, v$ ) //  $v$  is now on the frontier
    //  $u$  is now behind the frontier.
```

مثال : اجرای BFS روی یک گراف بدون جهت:

- پیکان‌ها به رأس در حال بازدید اشاره می‌کنند.
- یال‌هایی که با خطوط پررنگ رسم شده‌اند در زیرگراف پیشینی قرار دارند.
- خطوط چین به رأس‌های تازه کشف شده منتهی می‌شوند. این خطوط به صورت پررنگ رسم می‌شوند زیرا اکنون در زیرگراف پیشینی نیز قرار دارند.
- رأس‌هایی با حاشیه دوتایی کشف شده‌اند و در صف Q قرار دارند، در انتظار بازدید هستند.
- رأس‌هایی با حاشیه پررنگ کشف شده‌اند، از صف Q خارج شده‌اند و بازدید شده‌اند.



می‌توان نشان داد که Q شامل رأس‌هایی با مقادیر d است. (الگوی مقادیر در d بصورت زیر است)

$$k \quad k \quad k \quad \dots \quad k \quad k+1 \quad k+1 \quad \dots \quad k+1$$

- فقط ۱ یا ۲ مقدار.
- اگر ۲، با ۱ تفاوت دارند و همه کوچک‌ترین‌ها اول هستند.

از آنجا که هر رأس حداکثر یک بار مقدار d محدود دریافت می‌کند، مقادیر اختصاص داده شده به رأس‌ها به طور یکنواخت در طول زمان افزایش می‌یابند.

BFS ممکن است به همه رأس‌ها نرسد.

پیچیدگی زمانی الگوریتم $O(V + E)$.

- $O(V)$ زیرا هر رأس حداکثر یک بار در صف قرار می‌گیرد.
- $O(E)$ زیرا هر رأس حداکثر یک بار از صف خارج می‌شود و یال (u, v) فقط زمانی بررسی می‌شود که u از صف خارج شود. بنابراین، هر یال حداکثر یک بار اگر جهت‌دار باشد، حداکثر دو بار اگر بدون جهت باشد، بررسی می‌شود.

برای چاپ رأس‌های یک کوتاه‌ترین مسیر از s تا v :

```
PRINT-PATH( $G, s, v$ )
  if  $v == s$ 
    print  $s$ 
  elseif  $v.\pi == \text{NIL}$ 
    print “no path from”  $s$  “to”  $v$  “exists”
  else PRINT-PATH( $G, s, v.\pi$ )
    print  $v$ 
```